

סיכום מנהלים

סוכן AI שתפקידו למזער את כמות השאלות ולמקסם את ההסקה על הלקוח מתוך נתוני הקמפיין, נתוני גלישה (IP) ותשובות לשאלות קודמות (כלומר, פרסונליזציה של השאלון). הסוכן מעדכן state עם קליטת נתוני הקמפיין והגלישה, ולאחר מכן באופן איטרטיבי לאחר כל שאלה.

הגדרת הבעיה

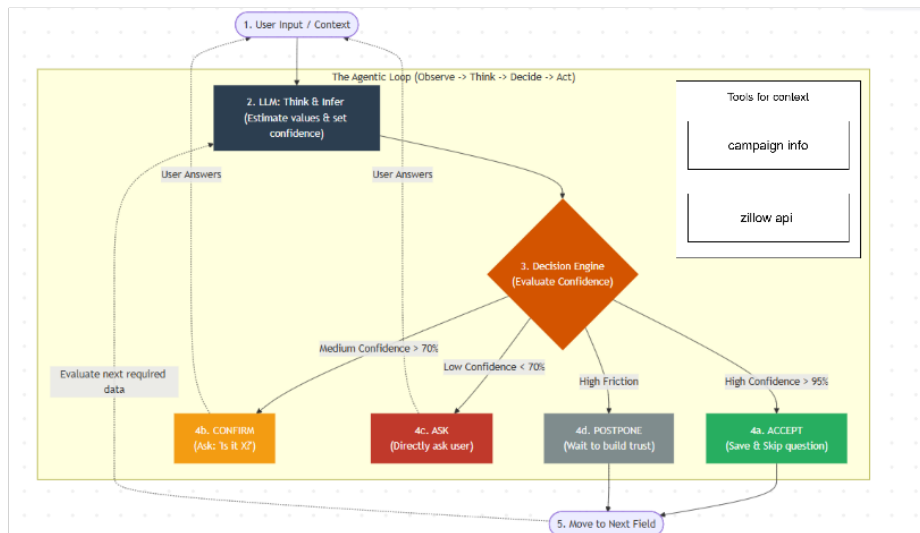
השאלון דורש מהמשתמש לספק מידע רב, רגיש, ומפורט הכולל נתונים פיננסיים הגורמים לנטישה מוגברת. נטישה זו מתרגמת לאובדן ישיר של ערך עסקי. ולכן, ככל שהסוכן יצליח להסיק בביטחון ובדיוק גבוהים יותר מידע על המשתמשים תוך כדי התהליך, ניתן יהיה לחסוך בשאלות ולצמצם נטישת משתמשים. בעצם, במצב הנוכחי לא מתרחשת למידה על המשתמש במהלך מילוי השאלון, ומטרתו של הסוכן היא ללמוד ולהתאים את השאלון באופן דינאמי תוך כדי מילוי. כלומר, הסוכן שואף להגדיל את שיעור המשתמשים המשלימים את השאלון במלואו, תוך שמירה על איכות המידע הנאסף.

עקרונות מנחים והנחות עבודה

1. ניתוב (Routing) טוב בין קמפיין לבין שאלון- השאלון קשור באופן ישיר לקמפיינים בנושא ההלוואות. הסוכן צריך להיות מותאם לוורטיקל שאליו בנוי השאלון (כמובן שניתן להתאים במהירות את הארכיטקטורה לוורטיקלים נוספים).
2. הצלחה בטירגוט של פייסבוק- לדוגמא כאשר מגיע משתמש דרך קמפיין של יוצאי צבא, הסוכן יניח שהוא יוצא צבא.
3. פתרון בעיית הנטישה- הבעיה שהסוכן פותר הוא נטישת משתמשים, והוא עושה את זה על חשבון דיוק. ניתן לבצע התאמה נכונה גם באמצעות מידע ברזולוציה נמוכה. לדוגמא, איננו זקוקים למספר הנקודות ב-Credit Score, אלא מספיק לנו לקבל תחום של המצב הכלכלי של המשתמש (poor, fair, rich) כדי לעשות התאמה נכונה. ולכן לא נדרש לשאול את המשתמש מה ה-Credit Score שלו, אלא לסווג אותו בהתאם לטווח בלבד.

ליבת הפתרון

1. מעגל אג'נטי (Agentic Loop) שמתבצע לאחר כל אינטראקציה מול המשתמש- שלביו: חילוף מאפיינים סמויים ואריזתם לתוך פרומפט - "נחש את הערכים עבור השדות החסרים, ותן לי ציון ביטחון (Confidence) בין 0 ל-1 עד כמה אתה בטוח בכל ניחוש".
2. מגנון ניתוב דטרמיניסטי שבוחר את השאלה הבאה בהינתן הביטחון של ה-LLM בפתרון- הסוכן משווה את ציון הביטחון שסיפק ה-LLM עבור כל שדה חסר למערכת של "ספי החלטה" (Thresholds) שהוגדרו מראש ב-Field Registry ובוחר את אסטרטגיית התשאול:
 - accept- רמת בטחון גבוהה, קבלת הערכה אוטומטית, אין צורך לשאול
 - confirm- רמת ביטחון בינונית, הסוכן יבקש מהמשתמש לאשר את הערך שמצאנו, ובכך יפחית את העומס הקוגניטיבי על המשתמש. לדוגמא "על בסיס המיקום שהזנת, נראה שאתה גר בבית צמוד קרקע, האם זה נכון?"
 - Ask- רמת בטחון נמוכה, הסוכן ישאל ישירות
3. להלן ארכיטקטורה של הסוכן:



התלבטויות והחלטות

1. מי קובע את השאלה הבאה? - אם ה-LLM קובע אותה, הוא עלול להיות בטוח בעצמו יותר על המידה, אם זה מנגנון קבוע מראש, אז אנחנו מאבדים מהגמישות והפשטות של הקוד. בחרתי לא לתת למודל שפה לבחור את השאלה הבאה ולהגדיר את האסטרטגיה בעצמי דרך Field Registry ופונקציה דטרמיניסטית המקבלת כקלט את רמת הבטחון של ה-LLM. כך אני מסתיר מה-LLM את ה-Threshold עבור דילוג על שאלה.
2. ניהול אינטראקציה בשפה חופשית- לא ראיתי לנכון לשלב ממשק מבוסס דיאלוג (chatbot). המטרה המרכזית היא לקדם את המשתמש לסיום השאלון, ובחירת תשובה מתוך 2-3 אפשרויות בלחיצת עכבר מהירה דורשת פחות מאמץ מהקלדת טקסט. בנוסף, חלופת השיחה בשפה חופשית תייצר Latency מתמשך בהמתנה לתגובות המודל, ותדרוש מעטפת אבטחה (Guardrails) מורכבת כדי למנוע ניצול לרעה של קלט פתוח לחלוטין (לדוגמה, משתמש המזין במטרת ההלוואה שהוא מעוניין בכסף לקניית נשק לביצוע פיגוע). בעתיד ניתן לבחון ב-A/B test האם עדיף סוכן דיאלוג על המצב הקיים.
3. ניהול זיכרון וקונטקסט של מודל השפה- ניהול ה-State מתבצע מחוץ ל-LLM. אבל ה-llm מקבל את התוצר שלו כל פעם מחדש. כיוון וה-state הוא יחסית חסכוני אז בחרתי לתת למודל את כל האינפורמציה מהמשתמש עד כאן בכל איטרציה. ייתכן וגם זה יקר מידי וזו דילמה שניתן לעצב נוכח ההגבלות המוצרות.
4. זמינות המערכת לשגיאות ממקורות חיצוניים- אם ממשק ה-api של Zillow או ה-LLM נופל, הסוכן עוצר את ההסקה החכמה ושואל את השאלות אחת אחר השנייה. מצד אחד, המשתמש לא יחווה מסך תקוע, מצד שני כנראה שרמת הנטישה תעלה בחזרה. במידה והפרויקט יוארך יש צורך ליצור מצב ביניים, שיביא לשיפור של המצב הקיים ללא תלות בגורמים חיצוניים. נקודה נוספת להמשך היא שימוש ב-llm on the machine ולא דרך קריאות api.

מתודולוגיה אלטרנטיבית

1. Tabular ML - לא לערב מודל שפה וליצור מודל פרדיקציה מנתוני עבר. היתרון הוא שקיפות גבוהה יותר, והחיסרון הוא שנדרש דאטה שאין ברשותנו. ייתכן ואפשרות חכמה תהיה להתחיל עם סוכן מבוסס LLM, ולאחר צבירת כמות מסוימת של תצפיות שימוש בכלים נוספים מבוססי ML קלאסי.
2. טכניקות של RL - להתייחס לכל שאלה כאל one step RL (contextual bandits) או על כל השאלון כ-eps-greedy. גם כאן נדרש שלב של אימון המודל שעלול להיות יקר (בהקשרי exploration, כלומר עלות עסקית, המודל צריך ללמוד על בסיס טעויות=נטישות). בהינתן שהמערכת צוברת משתמשים, ניתן לאמן על הלוגים שלהם RL, ולאחר התייצבות להכניס את הסוכן מבוסס RL לפרודקשן.

טרייד-אופים

1. Accuracy-recall/Accuracy- survival rate - ככל שהמשתמש מזין בעצמו מידע, הוא יותר מדויק וה-match בין לקוח למשתמש מובהק יותר. מנגד, ככל שהמשתמש מזין בעצמו יותר מידע, הוא מתעייף מהשאלון ועוזב אותו. מדובר בסוג של Accuracy-Recall Tradeoff. אם הרזולוציה הכי מדויקת אינה נדרשת על מנת לשמור על אותו דיוק ב-match, ייתכן וניתן להגדיל את ה-recall מבלי לפגוע ב-accuracy.
2. Latency מול State Adaptations - כל קריאה למודל שפה לוקחת מספר שניות, ולכן יש למזער את הקריאות למודל בתהליך. מצד שני, הסוכן צריך להתעדכן על פי התוצאות באופן שלא יהרוס לו את ההסקה בשלבים הבאים.
3. Latency-Explainability - על מנת להבין אילו תהליכי הסקה המודל עושה, ניתן לבקש ממנו פלט c.o.t או לחילופין לאלץ אותו להסיק על גבי איזשהו c.o.t. מנגד, זו הסקה איטית יותר שעלולה גם כן לגרום לנטישה.
4. Free writing-Choose - בחירה מבין אפשרויות מקלה על העומס הקוגניטיבי אך מביאה לאבדן דיוק בהתאמה. מנגד, באמצעות כתיבה בשפה חופשית המודל יכול לבצע את ההתאמה בין הטקסט לבין האפשרויות שלנו. שיקולי צד נוספים בהקשר זה הוא המחיר של הטוקנים ומגבלת תקציב שיש לקחת בחשבון, וההגנות שיש לשים על מודל שמקבל מידע בכתיבה חופשית.

אסטרטגיות שאלה

1. Infer, fallback ask - נסה להסיק, אם אתה בטוח אז אל תשאל בכלל. אבל אם אתה לא בטוח, תשאל. מתאים עבור פרמטר כמו military veteran, שאם הקמפיין נועד עבור veterans, אז התשובה היא וודאית. אילו הקמפיין הוא הלוואה לכל מטרה אז נדרש לשאול.
2. Infer and confirm, fallback ask - נסה להסיק, גם אם אתה בטוח, מדובר בנתון קריטי על שלא מאפשר להמשיך בחוסר ודאות, ולכן נבקש אישור מהמשתמש. אם לא הצלחת להסיק, תשאל מאפס.
3. Ask directly - יש חוסר ודאות גבוה בניחוש של המודל ולכן עדיף לשאול.

נקודות לשיפור

1. עדכון בייסיאני של הסתברויות לאחר כמות מסוימת של משתמשים - לאחר שהסוכן רץ זמן מסוים, אנחנו יכולים לבחון את ההנחות של הקמפיין, ולעדכן אותם data driven. אם עבדנו

- רק עם הפריור $p(\text{income} = \text{high} | \text{campaign})$ נוכל לעדכן את ההסתברות ל-
 $p(\text{income} = \text{high} | \text{campaign}, \text{LP_group_behavior})$.
2. המודל כרגע נשען על מידע מה-LLM בלבד, וככל שיתקדם הקמפיין כך נוכל לבצע עדכונים
עבור נתונים באמצעות שיטות similarity כמו knn למשל.
3. תקשורת מול המשתמש- כרגע השאלות מוגדרות היטב, ואין רספונסיביות של מודל שפה אל
המשתמש. מה שמשאיר את הניסוחים פשוטים ואולי לא רותמים מספיק.
4. שיקולי fairness- צריך לתחקר את המערכת ולראות שאין שימוש ב-confounders באופן לא
הוגן (למשל שליחת קבוצות אוכלוסייה מסוימת להלוואה מסוג מסוים באופן קבוע).